Week 7: RTOS Communication - Detailed Notes

Table of contents

1 Inter-Task Communication Overview

1.1 The Need for Communication

Tasks in an RTOS often need to:

1. **Share data** - Pass sensor readings, commands, status
2. **Coordinate actions** - Synchronize timing between tasks
3. **Protect resources** - Prevent conflicts on shared hardware

Without proper mechanisms, these interactions lead to **race conditions** - bugs that depend on unpredictable timing.

1.2 Communication Mechanisms

Mechanism	Purpose	Data Transfer?	Blocking?
Queue	Pass data between tasks	Yes (copies data)	Yes
Mutex	Protect shared resource	No	Yes
Semaphore	Signal events, count resources	No	Yes
Event flags	Wait for multiple events	No	Yes

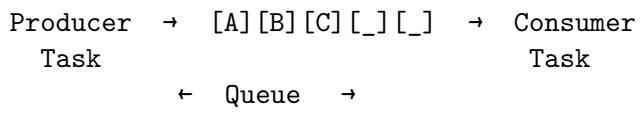
This week: Queues and Mutexes

2 Message Queues

2.1 Concept

A queue is a **FIFO (First-In-First-Out)** buffer managed by the RTOS:

- Producer tasks **send** items to the queue
- Consumer tasks **receive** items from the queue
- RTOS handles all synchronization automatically



2.2 Benefits Over Shared Variables

Problem	Shared Variable	Queue
Race condition	Possible	Protected
Partial data read	Possible	Atomic transfer
Timing coordination	Manual	Built-in blocking
Buffer overrun	No protection	Bounded size

2.3 Creating Queues

2.3.1 CubeMX Method

In FreeRTOS configuration → Queues tab:

- **Queue Name:** Identifier for the queue
- **Queue Size:** Number of items (depth)
- **Item Size:** Bytes per item

2.3.2 Manual Creation

```
// CMSIS-RTOS2 API
osMessageQueueId_t myQueue;
```

```
myQueue = osMessageQueueNew(
    10,                    // Queue depth (10 items)
    sizeof(uint16_t),      // Item size (2 bytes)
    NULL                   // Attributes (default)
);

// FreeRTOS native API
QueueHandle_t myQueue;
myQueue = xQueueCreate(10, sizeof(uint16_t));
```

2.4 Queue Operations

2.4.1 Sending Data

```
uint16_t data = 1234;

// CMSIS-RTOS2
osStatus_t status = osMessageQueuePut(
    myQueue,         // Queue handle
    &data,           // Pointer to data
    0,               // Message priority (unused)
    timeout          // How long to wait
);

// FreeRTOS native
BaseType_t result = xQueueSend(myQueue, &data, timeout);
```

2.4.2 Receiving Data

```
uint16_t received;

// CMSIS-RTOS2
osStatus_t status = osMessageQueueGet(
    myQueue,         // Queue handle
    &received,       // Where to store data
    NULL,            // Message priority output
    timeout          // How long to wait
);
```

```
// FreeRTOS native
BaseType_t result = xQueueReceive(myQueue, &received, timeout);
```

2.5 Timeout Options

Value	Behavior
0	Return immediately (non-blocking)
osWaitForever	Block until operation succeeds
N (ms)	Wait up to N milliseconds

2.5.1 Return Values

```
osStatus_t status = osMessageQueuePut(...);

switch (status) {
    case osOK:
        // Success
        break;
    case osErrorTimeout:
        // Queue full, timeout expired
        break;
    case osErrorResource:
        // Queue full (when timeout = 0)
        break;
    case osErrorParameter:
        // Invalid parameter
        break;
}
```

2.6 Queue Sizing Guidelines

Formula:

```
Queue depth  (Producer rate - Consumer rate) × Maximum burst time
```

Example: - Producer: 100 items/second - Consumer: 80 items/second average - Burst period: 500ms

```
Depth   (100 - 80) × 0.5 = 10 items minimum
```

Practical advice: Add 50% margin, so use 15 items.

2.7 Using Queues from ISR

Critical: Use `FromISR` variants in interrupt handlers:

```c
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    uint16_t value = HAL_ADC_GetValue(hadc);

    BaseType_t woken = pdFALSE;
    xQueueSendFromISR(dataQueue, &value, &woken);

    // Request context switch if higher-priority task woken
    portYIELD_FROM_ISR(woken);
}
```

Never use regular queue functions in ISR! They may block, which is forbidden in interrupts.

3 Mutexes

3.1 Concept

Mutex = Mutual Exclusion

A mutex ensures only one task accesses a shared resource at a time:

```
Task A:  [acquire][===use resource===][release]
                                          ↓
Task B:       [wait...]           [acquire][use][release]
```

3.2 Common Use Cases

- **Peripheral access:** UART, I2C, SPI shared by multiple tasks
- **Data structure:** Multi-field struct updated/read by tasks
- **File/memory:** Log file written by multiple tasks

3.3 Creating Mutexes

3.3.1 CubeMX Method

In FreeRTOS configuration → Mutexes tab:

- **Mutex Name:** Identifier
- **Type:** Normal or Recursive

3.3.2 Manual Creation

```
// CMSIS-RTOS2
osMutexId_t myMutex;
const osMutexAttr_t mutex_attr = {
    .name = "MyMutex",
    .attr_bits = osMutexPrioInherit,  // Enable priority inheritance
};
myMutex = osMutexNew(&mutex_attr);

// FreeRTOS native
SemaphoreHandle_t myMutex;
myMutex = xSemaphoreCreateMutex();
```

3.4 Using Mutexes

```
// Acquire (lock)
osStatus_t status = osMutexAcquire(myMutex, osWaitForever);

if (status == osOK) {
    // Critical section - safe to access shared resource
    printf("Protected output\n");

    // Release (unlock) - ALWAYS do this!
    osMutexRelease(myMutex);
}
```

3.4.1 Important Rules

1. **Always release what you acquire**
2. **Keep critical sections short**
3. **Don't call osDelay() while holding mutex**
4. **Never use mutex in ISR**

3.5 Priority Inversion Problem

Scenario: - Low-priority task holds mutex - High-priority task wants mutex, blocks - Medium-priority task runs, delaying low-priority

Result: High-priority task blocked by medium-priority task!

```
High:    [want mutex]  BLOCKED     [finally runs]
Medium:       [   RUNS     ]
Low:     [has mutex] preempted    [continues][release]
```

3.6 Priority Inheritance Solution

When high-priority task blocks on mutex: 1. Low-priority task **inherits** high priority 2. Low-priority completes quickly 3. High-priority runs immediately after release

```
// Enable priority inheritance
const osMutexAttr_t mutex_attr = {
    .attr_bits = osMutexPrioInherit,
};
```

3.7 Deadlock Prevention

Deadlock: Two tasks each hold what the other needs

```
// DEADLOCK EXAMPLE - DON'T DO THIS!
// Task A                    // Task B
osMutexAcquire(X);           osMutexAcquire(Y);
osMutexAcquire(Y);  // waits osMutexAcquire(X);  // waits
// Both blocked forever!
```

Prevention strategies:

1. **Lock ordering:** Always acquire mutexes in same order

2. **Timeout:** Use timeout instead of osWaitForever
3. **Single mutex:** Combine related resources under one mutex

4 Design Patterns

4.1 Producer-Consumer Pipeline

```
// Producer
void SensorTask(void *arg) {
    SensorData_t data;
    for(;;) {
        data = read_sensor();
        osMessageQueuePut(dataQueue, &data, 0, osWaitForever);
        osDelay(100);
    }
}

// Consumer
void ProcessTask(void *arg) {
    SensorData_t data;
    for(;;) {
        osMessageQueueGet(dataQueue, &data, NULL, osWaitForever);
        process(data);
    }
}
```

4.2 Protected Resource Access

```
osMutexId_t uartMutex;

void safe_print(const char *msg) {
    osMutexAcquire(uartMutex, osWaitForever);
    printf("%s", msg);
    osMutexRelease(uartMutex);
}

// Any task can call safely:
safe_print("Hello from Task A\n");
safe_print("Hello from Task B\n");
```

4.3 Command Dispatcher

```
typedef struct {
    uint8_t cmd;
    uint16_t param;
} Command_t;

// UI sends commands
void UITask(void *arg) {
    Command_t cmd = {CMD_START, 100};
    osMessageQueuePut(cmdQueue, &cmd, 0, 0);
}

// Worker executes commands
void WorkerTask(void *arg) {
    Command_t cmd;
    for(;;) {
        osMessageQueueGet(cmdQueue, &cmd, NULL, osWaitForever);
        execute(cmd);
    }
}
```

5 Quick Reference

5.1 Queue API Summary

Function	Purpose
osMessageQueueNew()	Create queue
osMessageQueuePut()	Send item
osMessageQueueGet()	Receive item
osMessageQueueGetCount()	Items in queue
osMessageQueueGetSpace()	Free space
osMessageQueueReset()	Empty queue
osMessageQueueDelete()	Destroy queue

5.2 Mutex API Summary

Function	Purpose
osMutexNew()	Create mutex
osMutexAcquire()	Lock mutex
osMutexRelease()	Unlock mutex
osMutexGetOwner()	Who holds it
osMutexDelete()	Destroy mutex

5.3 Decision Guide

Need	Use
Pass data between tasks	Queue
Protect shared peripheral	Mutex
Signal from ISR to task	Binary semaphore
Limit concurrent access	Counting semaphore
Wait for multiple conditions	Event flags

5.4 Common Mistakes

Mistake	Fix
Wrong queue item size	Use sizeof()
Forget mutex release	Release in all paths
Mutex in ISR	Use semaphore/queue
osWaitForever in ISR	Always use 0 timeout
Long critical section	Minimize locked code